

Use Cases: Local Produce Exchange

1. Introduction

This document is the high-level use cases for the Local Produce Exchange. It is based on the in-scope items from the [Team Charter](#) and [project requirements](#). Each use case stays at a high level. Deeper detail is left to the [user stories and their acceptance criteria](#), which are a different deliverable.

The roles used here are Guest, Member, Poster, Recipient, and Admin.

2. Actors and Roles

These are the roles the system defines. Poster and Recipient are not separate accounts. They are the role a Member plays for a given listing or request. The same person can be a Poster on one listing and a Recipient on another. In this document, an exchange means a request together with the coordination around it, so a phrase like "the exchange has status COMPLETED" refers to that request's status. A listing's quantity available is the amount the Poster enters when creating the listing. The listing's remaining quantity starts equal to the quantity available and goes down as the Poster approves requests. Some exception flows cover a situation that changed after the actor started, such as a listing that went inactive after the Member opened it. A suspended account cannot log in or take any member action; this rule applies in every use case.

- **Guest:** a person who holds an invite token but has not registered yet. A Guest can only register.
- **Member:** a registered, active user. A Member can browse, search, and filter listings, manage a profile, invite new members, post listings, submit requests, send messages, leave reviews, view notifications, and view a personal dashboard.
- **Poster:** the Member who owns a listing. A Poster creates and manages listings, handles each listing's request queue, messages the recipient, and marks a picked-up exchange complete.
- **Recipient:** the Member who submits a request for an item from a listing and, if approved, picks it up and confirms pickup. The poster reaches the recipient through the message thread.
- **Admin:** a privileged user who can suspend users, deactivate listings without deleting audit history, and generate basic reports.

3. Use Cases

3.1 Authentication and account

Use Case UC-01: Register with an invite token

- Primary actor: Guest
- Supporting actors: System (validates the token)
- Goal: Create a member account using a valid invite token so the Guest can join the community.

- Preconditions: The Guest holds an invite token. The token is valid and has not been used.
- Trigger: The Guest opens the registration page and submits account details with the token.
- Main success flow:
 - The Guest opens the registration page.
 - The Guest enters account details and the invite token.
 - The system checks that the token is valid and unused.
 - The system creates the member account.
 - The system marks the token as used.
- Alternate and exception flows:
 - Token is invalid or already used: the system rejects registration and creates no account.
 - Required account details are missing or malformed: the system rejects the submission and shows a validation error.
- Postconditions: A member account exists and the invite token is marked as used.
- Related user stories: US-01.

Use Case UC-02: Log in

- Primary actor: Member
- Supporting actors: System (checks credentials)
- Goal: Sign in to a member account to reach member features.
- Preconditions: The Member has a registered account that is not suspended.
- Trigger: The Member submits login credentials.
- Main success flow:
 - The Member opens the login page.
 - The Member enters credentials.
 - The system checks the credentials.
 - The system starts an authenticated session.
- Alternate and exception flows:
 - Credentials do not match: the system denies login and starts no session.
 - The account is suspended: the system denies login and tells the Member the account is suspended.
- Postconditions: The Member has an active authenticated session.
- Related user stories: US-02.

Use Case UC-03: Log out

- Primary actor: Member
- Supporting actors: System (ends the session)
- Goal: End the current session so no one else can use the account on this device.
- Preconditions: The Member is logged in.
- Trigger: The Member chooses to log out.
- Main success flow:
 1. The Member chooses to log out.
 2. The system ends the authenticated session.
 3. The system returns the Member to the public view.
- Alternate and exception flows:

- The session has already expired: the system simply returns the Member to the public view.
- Postconditions: The Member has no active session.
- Related user stories: US-03.

Use Case UC-04: Invite a new member

- Primary actor: Member
- Supporting actors: System (creates the invite token)
- Goal: Create and share an invite token so a new person can register.
- Preconditions: The Member is logged in and active. (Assumption: a Member may issue invite tokens.)
- Trigger: The Member chooses to invite a new person.
- Main success flow:
 - The Member chooses to create an invite.
 - The system generates a new, unused invite token.
 - The system shows the token so the Member can share it.
- Alternate and exception flows:
 - The member is suspended: the system denies the action and creates no token.
- Postconditions: A new, unused invite token exists.
- Related user stories: US-04.

Use Case UC-05: Manage member profile

- Primary actor: Member
- Supporting actors: System (saves the profile)
- Goal: View and update personal profile details.
- Preconditions: The Member is logged in.
- Trigger: The Member opens the profile page and edits details.
- Main success flow:
 1. The Member opens the profile page.
 2. The Member edits profile details.
 3. The system validates the changes.
 4. The system saves the updated profile.
- Alternate and exception flows:
 - A profile field is missing or malformed: the system rejects the change and shows a validation error.
 - A Member tries to edit another member's profile: the system denies the action.
- Postconditions: The Member's profile reflects the saved changes.
- Related user stories: US-05.

3.2 Discovery

Use Case UC-06: Browse, search, and filter active listings

- Primary actor: Member
- Supporting actors: System (returns matching listings)
- Goal: Find active listings of interest by browsing, searching, and filtering.

- Preconditions: The Member is logged in.
- Trigger: The Member opens the listings view or enters search and filter terms.
- Main success flow:
 - The Member opens the listings view.
 - The Member enters search terms or chooses filters such as category or dietary and allergen tags.
 - The system returns the active listings that match.
- Alternate and exception flows:
 - No listings match: the system shows an empty result.
- Postconditions: The Member sees the set of active listings that match.
- Related user stories: US-06.

Use Case UC-07: View listing details

- Primary actor: Member
- Supporting actors: System (returns the listing)
- Goal: See the full details of one listing before deciding to request it.
- Preconditions: The Member is logged in and the listing is active.
- Trigger: The Member selects a listing.
- Main success flow:
 1. The Member selects a listing.
 2. The system shows the listing's full details, including the remaining quantity, the tags, and the pickup window.
- Alternate and exception flows:
 - The listing is no longer active: the system tells the Member the listing is unavailable.
- Postconditions: The Member has seen the listing's full details.
- Related user stories: US-07.

Use Case UC-26: View another member's public profile

- Primary actor: Member
- Supporting actors: System (returns the profile)
- Goal: See another member's public profile and review history to assess trustworthiness before requesting/approving an exchange.
- Preconditions: The Member is logged in.
- Trigger: The Member selects another member's profile.
- Main success flow:
 1. The Member selects another member's profile.
 2. The system shows the member's display name and review history.
- Alternate and exception flows:
 - A member attempts to access this view without being logged in: system denies access.
 - A member views their own profile: the system shows their own public profile the same way another member would see it.
- Postconditions: The Member has seen the member's public profile and review history.
- Related user stories: US-30.

3.3 Request queue

Use Case UC-08: Submit a request for an item

- Primary actor: Recipient
- Supporting actors: Poster (owns the listing); System (checks quantity, queues the request, notifies the Poster)
- Goal: Request a specific quantity from an active listing so the request enters the listing's queue and is handled in order.
- Preconditions: The Recipient is a registered, active member. The listing is active and its remaining quantity is greater than zero.
- Trigger: The Recipient opens an active listing and chooses to request it.
- Main success flow:
 1. The Recipient opens an active listing.
 2. The Recipient enters the quantity they want.
 3. The system checks the amount against the listing's remaining quantity.
 4. The system creates a request with status REQUESTED and places it at the end of the listing's queue.
 5. The system notifies the Poster of the new request.
- Alternate and exception flows:
 - Requested amount is more than the remaining quantity: the system rejects the request and creates no request.
 - Requested amount is zero or negative: the system rejects it and shows a validation error.
 - The Recipient already has a pending request on this listing: the system prevents a duplicate request.
- Postconditions: A request exists with status REQUESTED, queued in the order it was received, and the Poster has been notified.
- Related user stories: US-08.

Use Case UC-09: View the request queue for a listing

- Primary actor: Poster
- Supporting actors: System (returns the queue)
- Goal: See the pending requests on the Poster's listing in the order they were received.
- Preconditions: The Poster owns the listing.
- Trigger: The Poster opens the request queue for one of their listings.
- Main success flow:
 1. The Poster opens one of their listings.
 2. The system shows the pending requests in the order received, oldest first, with the quantity requested and the remaining quantity.
- Alternate and exception flows:
 - A Member who does not own the listing tries to view its queue: the system denies access.
 - There are no pending requests: the system shows an empty queue.
- Postconditions: The Poster has seen the current request queue.
- Related user stories: US-09.

Use Case UC-10: Approve or deny the next request in the queue

- Primary actor: Poster
- Supporting actors: Recipient (the requester); System (checks quantity, updates status, notifies the Recipient)
- Goal: Handle a pending request on the Poster's listing by approving or denying it, keeping the queue order and preventing conflicts.
- Preconditions: The Poster owns the listing. At least one request has status REQUESTED. (Assumption: the Poster handles requests in the order received.)
- Trigger: The Poster chooses to approve or deny a pending request.
- Main success flow:
 1. The Poster opens the next pending request in the queue.
 2. The Poster chooses to approve or deny it.
 3. If approving, the system checks that the requested quantity is not more than the remaining quantity.
 4. The system sets the request status to APPROVED or DENIED and, when approving, reduces the remaining quantity.
 5. The system notifies the Recipient.
- Alternate and exception flows:
 - The requested quantity is more than the remaining quantity: the system rejects the approval.
 - A Member who does not own the listing tries to handle the request: the system denies access.
 - The request is not in REQUESTED status: the system rejects the action.
- Postconditions: The request has status APPROVED or DENIED, the remaining quantity reflects any approval, and the Recipient has been notified. A denial is final: the request cannot change status again.
- Related user stories: US-10.

Use Case UC-11: Withdraw a queued request

- Primary actor: Recipient
- Supporting actors: Poster (the other party); System (updates status, removes the request from the queue, notifies the Poster)
- Goal: Withdraw a pending request the Recipient no longer needs, removing it from the queue.
- Preconditions: The request has status REQUESTED and the Recipient owns it.
- Trigger: The Recipient chooses to withdraw their request.
- Main success flow:
 1. The Recipient opens their pending request.
 2. The Recipient chooses to withdraw it.
 3. The system sets the request status to CANCELLED and removes it from the queue.
 4. The system notifies the Poster.
- Alternate and exception flows:
 - The request is already APPROVED or DENIED: the system rejects the withdrawal.
 - A Member who is not the requester tries to withdraw: the system denies the action.
- Postconditions: The request has status CANCELLED, which is final, it is no longer in the queue, and the Poster has been notified. The remaining quantity is unchanged.
- Related user stories: US-11.

3.4 Coordination

Use Case UC-12: Send and read messages in the exchange thread

- Primary actor: Member (the Poster or the Recipient for the exchange)
- Supporting actors: System (stores and delivers the message)
- Goal: Send and read messages in the private thread that links a Poster and a Recipient for a request, so the two can coordinate the exchange.
- Preconditions: A request connects the Member as the Poster or the Recipient for that listing.
- Trigger: The Member opens the exchange thread and posts a message.
- Main success flow:
 - The Member opens the exchange thread for the request.
 - The Member writes and sends a message.
 - The system saves the message to the thread.
 - The system shows the message to both parties.
- Alternate and exception flows:
 - A Member who is not the Poster or the Recipient for that exchange tries to open the thread: the system denies access.
 - The message is empty: the system rejects it.
- Postconditions: The message is saved to the exchange thread and visible to both parties.
- Related user stories: US-12.

3.5 Listings

Use Case UC-13: Create a listing

- Primary actor: Poster
- Supporting actors: System (saves the listing)
- Goal: Post a new listing with the required details so other members can request the item.
- Preconditions: The Poster is a registered, active member.
- Trigger: The Poster chooses to create a listing.
- Main success flow:
 - The Poster chooses to create a listing.
 - The Poster enters the description, category, quantity available, dietary and allergen tags, and a pickup window.
 - The system validates the required details.
 - The system saves the listing and makes it active.
- Alternate and exception flows:
 - A required detail is missing or invalid: the system rejects the listing and shows a validation error.
 - The Poster also adds one or more optional photos: the system stores the photos with the listing.
 - The member is suspended: the system denies the action and creates no listing.
- Postconditions: A new active listing exists with the entered details.
- Related user stories: US-13, US-25.

Use Case UC-14: Edit a listing

- Primary actor: Poster
- Supporting actors: System (saves the changes)
- Goal: Update the details of an existing listing, including its photos.

- Preconditions: The Poster owns the listing.
- Trigger: The Poster chooses to edit one of their listings.
- Main success flow:
 - The Poster opens one of their listings to edit.
 - The Poster changes the details.
 - The system validates the changes.
 - The system saves the updated listing.
- Alternate and exception flows:
 - A changed detail is missing or invalid: the system rejects the change and shows a validation error.
 - The Poster adds, replaces, or removes optional photos: the system updates the listing's photos.
 - A Member tries to edit a listing they do not own: the system denies the action.
 - The member is suspended: the system denies the action and saves no change.
- Postconditions: The listing reflects the saved changes.
- Related user stories: US-14, US-25.

Use Case UC-15: Deactivate own listing

- Primary actor: Poster
- Supporting actors: System (hides the listing)
- Goal: Take a listing out of browsing so no new requests can be made on it.
- Preconditions: The Poster owns the listing and the listing is active.
- Trigger: The Poster chooses to deactivate one of their listings.
- Main success flow:
 1. The Poster opens one of their active listings.
 2. The Poster chooses to deactivate it.
 3. The system marks the listing inactive and hides it from browsing.
- Alternate and exception flows:
 - A Member tries to deactivate a listing they do not own: the system denies the action.
- Postconditions: The listing is inactive and no longer appears in browsing.
- Related user stories: US-15.

3.6 Pickup and completion

Use Case UC-16: Confirm pickup

- Primary actor: Recipient
- Supporting actors: Poster (the other party); System (updates status, notifies the Poster)
- Goal: Confirm that the Recipient has picked up the item so the exchange can move toward completion.
- Preconditions: The request has status APPROVED and the Recipient owns it. (Assumption: the Recipient confirms pickup.)
- Trigger: The Recipient chooses to confirm pickup.
- Main success flow:
 - The Recipient opens their approved request.
 - The Recipient chooses to confirm pickup.
 - The system changes the request status from APPROVED to PICKED_UP.

- The system notifies the Poster.
- Alternate and exception flows:
 - The request is not in APPROVED status: the system rejects the action and changes nothing.
 - A Member who is not the requester tries to confirm pickup: the system denies the action.
- Postconditions: The request has status PICKED_UP and the Poster has been notified.
- Related user stories: US-16.

Use Case UC-17: Complete an exchange

- Primary actor: Poster
- Supporting actors: Recipient (the other party); System (updates status, notifies the Recipient)
- Goal: Mark a picked-up exchange as completed so both parties can review it.
- Preconditions: The request has status PICKED_UP and the Poster owns the listing. (Assumption: the Poster marks the exchange complete.)
- Trigger: The Poster chooses to mark the exchange complete.
- Main success flow:
 1. The Poster opens the picked-up exchange.
 2. The Poster chooses to mark it complete.
 3. The system changes the request status from PICKED_UP to COMPLETED.
 4. The system notifies the Recipient.
- Alternate and exception flows:
 - The request is not in PICKED_UP status: the system rejects the action.
 - A Member who does not own the listing tries to complete the exchange: the system denies the action.
- Postconditions: The request has status COMPLETED and the Recipient has been notified.
- Related user stories: US-17.

3.7 Reviews

Use Case UC-18: Leave a rating and review after completion

- Primary actor: Member (acting as the reviewer)
- Supporting actors: System (saves the review and makes it visible to the reviewed party)
- Goal: Leave a short rating and review of the other party after an exchange is completed, so each party gets feedback on how the exchange went.
- Preconditions: The exchange has status COMPLETED. The Member is the Poster or the Recipient for that exchange and has not already reviewed the other party.
- Trigger: The Member chooses to leave a review for the completed exchange.
- Main success flow:
 - The Member opens the completed exchange.
 - The Member enters a rating and a short review of the other party.
 - The system saves the review and links it to the completed exchange.
 - The system makes the saved review visible to the reviewed party.
- Alternate and exception flows:
 - The exchange is not COMPLETED: the system rejects the review and saves nothing.

- A Member who is not a participant tries to review: the system denies access.
- The Member has already reviewed the other party for this exchange: the system rejects the duplicate review.
- Postconditions: The review is saved, linked to the completed exchange, and visible to the reviewed party.
- Related user stories: US-18.

Use Case UC-19: View reviews for a completed exchange

- Primary actor: Member
- Supporting actors: System (returns the reviews)
- Goal: View the reviews left for a completed exchange, including a review left about the viewing member.
- Preconditions: The exchange has status COMPLETED. The Member is the Poster or the Recipient for that exchange.
- Trigger: The Member opens the reviews for a completed exchange.
- Main success flow:
 1. The Member opens a completed exchange.
 2. The system shows the reviews linked to that exchange, including any review left about the viewing member.
- Alternate and exception flows:
 - No review has been left yet: the system shows that there are no reviews yet.
 - A Member who is not a participant tries to view the reviews: the system denies access.
- Postconditions: The Member has seen the reviews for the completed exchange.
- Related user stories: US-19.

3.8 Notifications and dashboard

Use Case UC-20: View status notifications

- Primary actor: Member
- Supporting actors: System (returns the notifications)
- Goal: Open and read in-app notifications about exchange status changes.
- Preconditions: The Member is logged in.
- Trigger: The Member opens their notifications.
- Main success flow:
 - The Member opens their notifications.
 - The system shows the Member's notifications, newest first.
 - The Member reads a notification and can open the related exchange.
- Alternate and exception flows:
 - There are no notifications: the system shows an empty list.
- Postconditions: The Member has seen their current notifications.
- Related user stories: US-20.

Use Case UC-21: View my dashboard and activity overview

- Primary actor: Member

- Supporting actors: System (gathers the member's activity)
- Goal: See a single overview of the Member's own activity: active listings, incoming requests, outgoing requests, and exchange history.
- Preconditions: The Member is logged in.
- Trigger: The Member opens their dashboard.
- Main success flow:
 1. The Member opens their dashboard.
 2. The system gathers the Member's active listings, incoming requests, outgoing requests, and exchange history.
 3. The system shows these grouped by status, with links to act on each item.
- Alternate and exception flows:
 - The Member has no activity yet: the system shows empty groups.
- Postconditions: The Member has seen their current activity overview.
- Related user stories: US-21.

3.9 Admin

Use Case UC-22: Suspend a user

- Primary actor: Admin
- Supporting actors: System (updates the account)
- Goal: Suspend a member account so it can no longer log in or take member actions.
- Preconditions: The Admin is logged in with admin rights. The target account exists.
- Trigger: The Admin chooses to suspend a user.
- Main success flow:
 - The Admin opens the target user account.
 - The Admin chooses to suspend it.
 - The system marks the account suspended.
- Alternate and exception flows:
 - A non-admin user tries to suspend a user: the system denies the action.
- Postconditions: The account is suspended and cannot log in or take member actions.
- Related user stories: US-22.

Use Case UC-23: Deactivate a listing as admin

- Primary actor: Admin
- Supporting actors: System (hides the listing, keeps audit history)
- Goal: Hide a listing from browsing without deleting its audit history.
- Preconditions: The Admin is logged in with admin rights. The listing exists.
- Trigger: The Admin chooses to deactivate a listing.
- Main success flow:
 1. The Admin opens the target listing.
 2. The Admin chooses to deactivate it.
 3. The system hides the listing from browsing and keeps its audit history.
- Alternate and exception flows:

- A non-admin user tries to deactivate a listing as admin: the system denies the action.
- Postconditions: The listing is hidden from browsing and its audit history is kept.
- Related user stories: US-23.

Use Case UC-24: Generate basic reports

- Primary actor: Admin
- Supporting actors: System (computes and returns the report)
- Goal: Generate a basic report about system activity.
- Preconditions: The Admin is logged in with admin rights.
- Trigger: The Admin chooses to generate a report.
- Main success flow:
 1. The Admin chooses a report to generate.
 2. The system computes the report.
 3. The system shows the report to the Admin.
- Alternate and exception flows:
 - A non-admin user tries to generate a report: the system denies the action.
- Postconditions: The Admin has seen the generated report.
- Related user stories: US-24.

Use Case UC-25: View member profile as admin

- Primary actor: Admin
- Supporting actors: System (returns full account details)
- Goal: View a member's full account details in order to take administrative action such as suspending or reinstating the account.
- Preconditions: The Admin is logged in with admin rights.
- Trigger: The Admin searches for or selects a member account.
- Main success flow:
 1. The Admin searches for a member by name or email.
 2. The system returns matching members with their account status.
 3. The Admin selects a member.
 4. The system shows the member's full account details, including account status, suspension history, and exchange history
- Alternate and exception flows:
 - No members match the search: system shows empty result/
 - Non-admin attempts to access view: system denies access
 - The Admin selects a suspended account: the system shows the account details with suspended status and makes the reinstate action available.
- Postconditions: The admin has seen the member's full account details and can proceed to suspend or reinstate the account.
- Related user stories: US-29

4. Assumptions and open questions

The team has not yet approved the items below. Each must be reviewed and either confirmed or changed before the requirements are locked. They follow the course guidance to record open questions and assumptions for the review packet.

ID	Type	Item	Affected use cases	Status
-----------	-------------	-------------	---------------------------	---------------
